Shared AI33 API and Music Library: Rules for Every Hark

Repo: https://github.com/zaimzon021/Music-Library

These rules are mandatory for every person and every AI (every Hark) that uses the shared AI33 API key or this repository.

The goal is the **best possible video quality** with **no wasted credits**. Spending credits is allowed whenever it makes a video better. It is **never** allowed for duplicates, repeats, or work the library already covers.

Rule 1. Library first, always

Before you generate **any** music or sound effect:

1. `git pull` to get the newest library.
2. Search it:

```
python3 tools/lib.py search "dark horror music box dread" --type music
python3 tools/lib.py search "giant monster roar" --type sfx
```

3. Listen to or read the closest results. If one fits, **use it**. That costs nothing.

Rule 2. Generate only when the library truly doesn't have it

You may generate new music or SFX **only if**:

- nothing in the library fits, **or**
- what's there is clearly a different mood, style, instrument, or energy from what the video needs.

Never generate something that is too similar to a file already in the library. Check before every generation:

```
python3 tools/lib.py similar --prompt "your planned prompt" --type music
```

- `REUSE, DO NOT GENERATE` = something very similar exists. Use it.
- `OK TO GENERATE` = go ahead (Rule 3 applies).

"Too similar" means it would sound like basically the same track or sound. Two tracks that share a genre but differ clearly in mood, tempo, instruments, or structure are **not** too similar, and making the new one is fine when the video needs it.

`tools/ai33.py` runs this check automatically for `sfx()` and `music()`, and refuses when the library already has a close match.

Rule 3. Everything you generate must be reusable

- Describe **moods and styles**, not one video's content. Good: "ominous deep-sea suspense, sonar pings, slow build, huge reveal hit". Bad: "music for the Titanoboa video at 0:23".
- Music: always **instrumental**, a full cue with intro, build, and climax so anyone can cut any section.
- SFX: clean single sounds (one roar, one impact, one whoosh) with no music or voice on them.
- Video-specific sound is allowed when the quality truly needs it. Still name and tag it so others can find and reuse it.

Rule 4. Quality comes first, but never pay twice

- If a new track, SFX, or voice take will clearly make the video better, spend the credits. That's allowed.
- If spending doesn't improve quality, don't do it.
- Never re-generate something you or anyone else already made. Reuse it.
- Never run "just to see" generations of full scripts or full songs. Test with a short line when you need to hear something.

Rule 5. Every new sound goes into the repo (mandatory)

Every music track or sound effect you generate is added to this repo, the same day, with a library entry. No exceptions. This is how the library grows and how everyone saves credits.

```
python3 tools/lib.py add new_roar.mp3 --type sfx --category creatures \
   --slug lizard-screech --tags screech,monster,high-pitched \
   --prompt "the exact prompt you used" \
   --generator "ElevenLabs SFX v3 via AI33" --by "your-channel-name"
git add -A
git commit -m "add sfx: creatures/lizard-screech (your-channel)"
git push
```

- Suno returns **2 tracks per call**: add **both** (`-a`, `-b`).
- Put the exact prompt in `--prompt`. It makes the duplicate check work.
- Categories (folders): reuse an existing one when it fits. Make a new one only for a truly new mood.
- Run `python3 tools/lib.py validate` before pushing.
- Don't delete or rename other people's files. Fix tags if they're wrong.

Rule 6. Voiceovers: patch first, full re-voice only as a fallback

1. **Lock the script before any TTS.** Fact-check it, finish the wording, check the length. Every re-voice costs credits.
2. **One TTS job per full script.** Never line by line, never sentence by sentence.
3. **If only one part is wrong** (for example, the hook or the first 30-60 seconds is weak but the rest is good):
   - **Do not re-voice the whole script.**
   - Voice **only the replacement part** and splice it in:
     ```
     python3 tools/voice_patch.py --voice voice.mp3 --words words.json \
        --replace-until "last words of the part to replace" \
        --new-text "The new hook." --voice-id <same voice> \
     ```

```
--model <same model> --speed <same speed> \
--out voice_new.mp3 --out-words words_new.json
```

For a part in the middle, add `--replace-from "first words of the part"`.

- The tool cuts at sentence ends inside natural pauses, matches loudness, crossfades, and re-times the captions.
- Always use the **same voice, model, and speed** as the original.
- **Listen to the join.** Then re-render the video (rendering is free).

4. **Full re-voice only if the patch doesn't work.** That means the tool prints `FULL REGEN ADVISED`, or the join audibly doesn't match (tone, pace, or energy jump), or the patch would leave only the new part playing. Only then re-voice the whole script.

Rule 7. Images

Use free sources first (wikis, Wikimedia Commons, public-domain stills). Use AI image generation (~2,574 credits each) only when no real image can do the job well, such as a thumbnail concept that doesn't exist.

Rule 8. Credits and budget

- Check the balance before a batch: `python3 -c "from tools.ai33 import credits; print(credits())"`.
- Keep a simple log of what you spent per video.
- Recommended: each person uses their own AI33 key so nobody's jobs drain someone else's credits. On a shared key, stick to your agreed daily share.

Approximate credit costs (AI33, Nov 2026)

Job	Credits
Voiceover, ElevenLabs v3, ~60 s Short	~1,100-1,300
Sound effect	~100-300
Suno music (1 call = 2 tracks)	3,600
AI image	~2,574

Reusing a library file: **0**.

Paste this into your Hark as a specialty

In Hark, say: **"Save this as a specialty called shared-sound-library and follow it for every video"**, then paste the text below.

```
SHARED SOUND LIBRARY + AI33 RULES (mandatory for every video)

Repo: https://github.com/zaimzon021/Music-Library  (clone it into the workspace, run
   `git pull` before each video)
Full rules: RULES.md in the repo. Read it on first use.

1. LIBRARY FIRST. Before generating any music or sound effect, search the repo:
   python3 tools/lib.py search "<mood/sound words>" --type music|sfx
   Use a library file whenever one fits. That costs zero credits.
2. GENERATE ONLY WHEN NEEDED. Generate new music or SFX only when nothing in the
   library fits, or what's there is clearly different from what the video needs.
   Before every generation, run:
   python3 tools/lib.py similar --prompt "<planned prompt>" --type music|sfx
   If it says REUSE, DO NOT GENERATE, use that file. Never generate something too
   similar to an existing file. Similar genre with a clearly different mood, tempo or
   instruments is fine.
3. MAKE IT REUSABLE. Prompts describe moods and styles, not one video. Music is
   instrumental and full-length with intro, build and climax. SFX are single clean
   sounds.
4. QUALITY FIRST, NO WASTE. Spend credits whenever it clearly improves the video.
   Never spend on duplicates, repeats, or "just to see" full generations.
5. PUSH EVERYTHING NEW. Every music track (both Suno outputs) and every SFX you
   generate gets added the same day:
   python3 tools/lib.py add <file> --type ... --category ... --slug ... --tags ...
   --prompt "<exact prompt>" --generator ... --by "<channel>"
   python3 tools/catalog.py && python3 tools/lib.py validate
   git add -A && git commit -m "add <type>: <category>/<slug> (<channel>)" && git push
6. VOICE. Lock and fact-check the script before TTS. Use one TTS job per full script,
   never line by line. If only one part is wrong (for example, the hook), DON'T re-
   voice everything. Voice only that part and splice it with tools/voice_patch.py
   (same voice, model, speed), then listen to the join. Re-voice the full script only
   if the tool prints FULL REGEN ADVISED or the join audibly doesn't match.
7. IMAGES. Use free real images first. Use AI image generation only when no real image
   works.
8. CREDITS. Check the balance before a batch and log what each video spent.
```